

North South University

Department of Electrical and Computer Engineering

CSE445: Machine Learning

Assignment#3: Building an Autonomous Local LLM Machine Learning Agent in Windows WSL

Course	CSE445 — Machine Learning	Semester / Section	Section 6 (MW 2:40 PM - 4:10 PM)
Instructor	Dr. Mohammad Abdul Qayum (MAQm)	Weightage	Assignment / Project Component
Course Outcomes	CO1 (Understand ML Algorithms), CO2 (Apply Statistical & Mathematical Analysis), CO3 (Implement ML Project with Engineering Precision)		
Primary Reference	<i>Machine Learning with PyTorch and Scikit-Learn</i> (Sebastian Raschka, Yuxi Liu, Vahid Mirjalili)		

1. Assignment Overview and Objectives

Modern machine learning workflows increasingly rely on intelligent orchestration, automated model selection, and agentic workflows. In this assignment, students will design and implement a fully local, privacy-preserving **Autonomous Machine Learning Agent** running on Windows Subsystem for Linux (WSL2).

The agent will leverage a quantized open-source Large Language Model (e.g., Llama-3.2, Mistral, or Phi-3) served via Ollama in WSL to reason, select tools, and orchestrate classical ML pipelines (Scikit-Learn) and deep learning models (PyTorch) based on user instructions.

Key Learning Outcomes:

- **Local LLM Inference & Architecture:** Set up and manage local quantized LLMs inside a Linux environment (Ubuntu on WSL2) without reliance on external paid APIs.
- **Agentic Reasoning Patterns:** Implement the ReAct (Reason + Act) loop from foundational principles to enable structured tool-calling.
- **ML Workflow Automation:** Equip the LLM agent with custom Python tools that invoke Scikit-Learn estimators, PyTorch neural networks, evaluation metrics, and exploratory data analysis routines.
- **Robust Error Handling & Verification:** Handle hallucinated inputs, parse structured

execution traces, and ensure reliable execution of ML tasks.

2. System Architecture & ReAct Agent Design

The autonomous agent operates in a cyclic loop of **Thought**, **Action**, **Action Input**, and **Observation** before producing a **Final Answer**.

Component	Technology	Role in Assignment
OS / Subsystem	Windows WSL2 (Ubuntu 22.04 / 24.04 LTS)	Provides native Linux kernel execution with direct GPU passthrough (CUDA).
Inference Engine	Ollama (Local REST API on port 11434)	Runs quantized 4-bit/8-bit models (e.g., Llama 3.2 3B or Mistral 7B) locally.
Agent Logic	Python 3.10+ Custom ReAct Controller	Maintains conversation state, prompts the local model, parses actions, and executes functions.
ML Frameworks	PyTorch, Scikit-Learn, Pandas, NumPy	Executes data preprocessing, model fitting, metric evaluations, and predictions.

3. Step-by-Step Hands-On Tutorial

Step 3.1: Environment Setup in Windows WSL2

Open Windows PowerShell (Run as Administrator) and verify or install WSL with Ubuntu:

```
# In Windows PowerShell:  
wsl --install -d Ubuntu-22.04  
wsl --set-default-version 2
```

Once inside your Ubuntu terminal in WSL, update packages and install core build tools and Python:

```
# In WSL Ubuntu Bash:  
sudo apt update && sudo apt upgrade -y  
sudo apt install -y python3-pip python3-venv curl build-essential git
```

```
# Create dedicated workspace
mkdir -p ~/cse445_agent && cd ~/cse445_agent
python3 -m venv venv
source venv/bin/activate
```

Step 3.2: Install and Serve Local LLM via Ollama

Install Ollama inside WSL, start the background service, and pull a lightweight instruction-tuned model:

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Start Ollama service in the background (or in a separate terminal tab)
ollama serve &

# Pull a lightweight model (Llama 3.2 3B or Mistral 7B)
ollama pull llama3.2:3b
```

Step 3.3: Install Python Dependencies

Install PyTorch (with CPU or CUDA support) and Scikit-Learn as referenced in the course textbook:

```
pip install --upgrade pip
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
pip install scikit-learn pandas numpy requests pydantic
```

Step 3.4: Defining the Machine Learning Tools

Create a file named `ml_tools.py` that encapsulates standard Machine Learning operations (Decision Trees, Logistic Regression, PyTorch MLP, and Data Explorers):

```
# ml_tools.py
import json
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris, load_wine, load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import torch
import torch.nn as nn
import torch.optim as optim
```

```
DATASETS = {
    "iris": load_iris,
    "wine": load_wine,
    "breast_cancer": load_breast_cancer
}
```

```
def load_dataset_summary(dataset_name: str) -> str:
    """Loads a standard benchmark dataset and returns summary statistics."""
    name = dataset_name.lower().strip()
    if name not in DATASETS:
        return json.dumps({"error": f"Unknown dataset '{name}'. Options: {list(DATASETS.keys())}"})
```

```
    data = DATASETS[name]()
    df = pd.DataFrame(data.data, columns=data.feature_names)
    df['target'] = data.target
```

```
    summary = {
        "dataset": name,
        "n_samples": df.shape[0],
        "n_features": len(data.feature_names),
        "feature_names": data.feature_names,
        "classes": [str(c) for c in np.unique(data.target)],
        "missing_values": int(df.isnull().sum().sum())
    }
    return json.dumps(summary)
```

```
def train_sklearn_model(dataset_name: str, model_type: str, test_size: float = 0.2) -> str:
    """Trains a Scikit-Learn model (decision_tree, logistic_regression, random_forest) on a dataset."""
    name = dataset_name.lower().strip()
    if name not in DATASETS:
        return json.dumps({"error": f"Dataset '{name}' not found."})
```

```
    data = DATASETS[name]()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=test_size, random_state=42, stratify=data.target
    )
```

```
    model_type = model_type.lower().strip()
    if model_type == "decision_tree":
        clf = DecisionTreeClassifier(max_depth=4, random_state=42)
    elif model_type == "logistic_regression":
        clf = LogisticRegression(max_iter=1000, random_state=42)
    elif model_type == "random_forest":
        clf = RandomForestClassifier(n_estimators=50, random_state=42)
    else:
        return json.dumps({"error": f"Unsupported model '{model_type}'."})
```

```

clf.fit(X_train, y_train)
preds = clf.predict(X_test)
acc = accuracy_score(y_test, preds)
cv_scores = cross_val_score(clf, data.data, data.target, cv=5)

```

```

return json.dumps({
    "model": model_type,
    "dataset": name,
    "test_accuracy": round(acc, 4),
    "cv_mean_accuracy": round(float(cv_scores.mean()), 4),
    "cv_std": round(float(cv_scores.std()), 4)
})

```

```

def train_pytorch_mlp(dataset_name: str, hidden_dim: int = 32, epochs: int = 50, lr: float = 0.01) -> str:

```

```

    """Trains a PyTorch Multilayer Perceptron on the selected classification dataset."""
    name = dataset_name.lower().strip()
    if name not in DATASETS:
        return json.dumps({"error": f"Dataset '{name}' not found."})

```

```

    data = DATASETS[name]()
    X_train, X_test, y_train, y_test = train_test_split(
        data.data, data.target, test_size=0.2, random_state=42, stratify=data.target
    )

```

```

    # Feature standardization
    mean, std = X_train.mean(axis=0), X_train.std(axis=0) + 1e-7
    X_train = (X_train - mean) / std
    X_test = (X_test - mean) / std

```

```

    num_features = X_train.shape[1]
    num_classes = len(np.unique(data.target))

```

```

    X_t = torch.tensor(X_train, dtype=torch.float32)
    y_t = torch.tensor(y_train, dtype=torch.long)
    X_val_t = torch.tensor(X_test, dtype=torch.float32)
    y_val_t = torch.tensor(y_test, dtype=torch.long)

```

```

    model = nn.Sequential(
        nn.Linear(num_features, hidden_dim),
        nn.ReLU(),
        nn.Linear(hidden_dim, num_classes)
    )

```

```

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

```

```

for epoch in range(epochs):
    optimizer.zero_grad()
    out = model(X_t)
    loss = criterion(out, y_t)
    loss.backward()
    optimizer.step()

with torch.no_grad():
    test_out = model(X_val_t)
    test_preds = torch.argmax(test_out, dim=1)
    acc = (test_preds == y_val_t).float().mean().item()

return json.dumps({
    "framework": "PyTorch",
    "dataset": name,
    "hidden_dim": hidden_dim,
    "epochs": epochs,
    "final_loss": round(float(loss.item()), 4),
    "test_accuracy": round(acc, 4)
})

# Registry mapping tool names to callable Python functions
AVAILABLE_TOOLS = {
    "load_dataset_summary": load_dataset_summary,
    "train_sklearn_model": train_sklearn_model,
    "train_pytorch_mlp": train_pytorch_mlp
}

```

Step 3.5: Implementing the Local LLM ReAct Agent Controller

Create a file named `react_agent.py`. This script defines the agent prompt, queries Ollama via HTTP REST, parses the action output, calls the tool, and iterates until Final Answer: is achieved:

```

# react_agent.py
import re
import json
import requests
from ml_tools import AVAILABLE_TOOLS

OLLAMA_URL = "http://127.0.0.1:11434/api/generate"
MODEL_NAME = "llama3.2:3b"

SYSTEM_PROMPT = """You are an expert Autonomous Machine Learning Assistant.
You solve machine learning problems by thinking step-by-step and invoking external tools.

```

You have access to the following tools:

1. `load_dataset_summary(dataset_name: str)` -> JSON summary of dataset (iris, wine, breast_cancer).
2. `train_sklearn_model(dataset_name: str, model_type: str, test_size: float = 0.2)` -> JSON test/CV scores (model_type: decision_tree, logistic_regression, random_forest).
3. `train_pytorch_mlp(dataset_name: str, hidden_dim: int = 32, epochs: int = 50, lr: float = 0.01)` -> JSON PyTorch training and evaluation results.

To use a tool, you MUST strictly use this format:

Thought: Describe your reasoning about what to do next.

Action:

Action Input: {"param_name": "value"}

When you have received the observation and are ready to provide the complete answer to the user, format your output as:

Thought: I have gathered all necessary experimental data.

Final Answer:

Begin!

```
"""
```

```
def query_local_llm(prompt: str) -> str:
    """Queries the local Ollama instance running in WSL."""
    payload = {
        "model": MODEL_NAME,
        "prompt": prompt,
        "stream": False,
        "options": {
            "temperature": 0.1,
            "stop": ["Observation:"]
        }
    }
    response = requests.post(OLLAMA_URL, json=payload)
    if response.status_code != 200:
        raise RuntimeError(f'Ollama error: {response.text}')
    return response.json().get("response", "")

def run_agent_loop(user_query: str, max_iterations: int = 6):
    print(f"\n=====")
    print(f"USER QUERY: {user_query}")
    print(f"=====\\n")

    prompt = f'{SYSTEM_PROMPT}\\nUser Query: {user_query}\\n'

    for step in range(1, max_iterations + 1):
        print(f"\n--- Step {step} ---")
        llm_output = query_local_llm(prompt)
        print(llm_output)
```

```
prompt += llm_output
```

```
if "Final Answer:" in llm_output:  
    print("\n>>> Task Completed Successfully!")  
    break
```

```
action_match = re.search(r"Action:\s*([a-zA-Z0-9_]+)", llm_output)  
input_match = re.search(r"Action Input:\s*(\{.*?\})", llm_output, re.DOTALL)
```

```
if action_match and input_match:  
    tool_name = action_match.group(1).strip()  
    raw_input = input_match.group(1).strip()
```

```
try:  
    kwargs = json.loads(raw_input)  
except json.JSONDecodeError:  
    observation = "Observation: Error parsing Action Input as JSON."  
    prompt += f"\n{observation}\n"  
    print(observation)  
    continue
```

```
if tool_name in AVAILABLE_TOOLS:  
    try:  
        tool_res = AVAILABLE_TOOLS[tool_name](**kwargs)  
        observation = f"\nObservation: {tool_res}\n"  
    except Exception as e:  
        observation = f"\nObservation: Tool execution error: {str(e)}\n"  
else:  
    observation = f"\nObservation: Tool '{tool_name}' not recognized.\n"
```

```
    print(observation)  
    prompt += observation  
else:  
    prompt += "\nObservation: Please respond with an Action and Action Input or a Final  
Answer.\n"
```

```
if __name__ == "__main__":  
    test_task = "Analyze the breast_cancer dataset, train a Random Forest and a PyTorch  
MLP on it, compare their accuracies, and recommend the best model for clinical screening."  
    run_agent_loop(test_task)
```


4. Student Assignment Tasks

Students must complete three progressive implementation tasks, integrating concepts from *Machine Learning with PyTorch and Scikit-Learn*:

Task 1: Environment & Baseline ReAct Engine (25 Marks)

- Configure WSL2, Ollama, and Python virtual environment with full GPU/CPU PyTorch support.
- Implement the baseline ReAct loop and verify autonomous single-tool and multi-tool execution traces.

Task 2: Advanced ML Tool Expansion (40 Marks)

Expand `ml_tools.py` by implementing three additional tools based on textbook topics:

1. **Hyperparameter Tuning Tool:** Integrate GridSearchCV or RandomizedSearchCV for Support Vector Classifiers (SVC / Kernel SVM) and Decision Trees.
2. **Feature Selection & Dimensionality Reduction Tool:** Implement PCA (Principal Component Analysis) and sequential feature selection pipelines.
3. **Deep PyTorch Classifier with Regularization:** Implement a configurable PyTorch module featuring Dropout, BatchNorm, and customized learning rate schedulers.

Task 3: Self-Correction & Model Comparison Benchmark (35 Marks)

- Add self-healing logic to the agent controller: If a tool throws a shape mismatch, parameter error, or NaN loss, the agent must catch the observation, reason about the failure, and re-attempt execution with corrected parameters.
- Formulate a comprehensive prompt where the agent autonomously evaluates 3 algorithms across 2 datasets, performs cross-validation, and writes a Markdown experimental summary table.

5. Submission Guidelines & Deliverables

- **GitHub Repository Link:** Containing well-documented code (`ml_tools.py`, `react_agent.py`, `benchmark_runner.py`, `requirements.txt`).
- **Execution Logs:** A recorded terminal log (or exported text file) demonstrating at least 3 multi-step agent reasoning traces with tool execution.
- **Technical Report (PDF / Markdown):** A 3–5 page report explaining:
 - Local LLM architecture, prompt engineering techniques, and latency benchmarks in WSL2.
 - Mathematical comparison of models evaluated by the agent (linking with CO1 and CO2).
 - Architectural diagram of the agent controller loop and tool registry.

6. Grading Rubric (Mapped to Course Outcomes)

Evaluation Criteria	Marks	Target CO	Performance Indicators
WSL & Local LLM Setup	15	CO3	Successful execution of quantized model inside WSL environment via Ollama.
ReAct Loop & Action Parsing	25	CO3	Accurate parsing of Thought/Action/Input patterns and robust execution without manual intervention.
PyTorch & Scikit-Learn Tool Implementation	30	CO1, CO3	Correct implementation of estimators, neural network architectures, validation splits, and metric summaries.
Statistical Analysis & Agent Reasoning Quality	20	CO2	Sound statistical interpretation of cross-validation results, variance, and model trade-offs in Final Answers.
Report & Engineering Quality	10	CO3	Clarity of documentation, structured repo, reproducibility of instructions.
Total	100	—	Complete evaluation score.